Enterprise Disaster Recovery & Backup Architecture

Project Description:
This project focuses on designing and implementing a robust Enterprise Disaster Recovery (DR) and Backup Architecture on AWS to ensure high availability, data protection, and business continuity for a financial services organization.

The solution is designed to meet strict service-level requirements, including a Recovery Point Objective (RPO) of 1 hour and a Recovery Time Objective (RTO) of 4 hours, ensuring minimal data loss and rapid recovery during failures or disasters.

The architecture leverages key AWS services such as Amazon EC2 for compute resources, Amazon EBS for persistent block storage, and Amazon EFS for scalable shared storage. Backup and recovery operations are centralized using AWS Backup.

To support hybrid data integration, AWS Storage Gateway is used to connect on-premises systems with cloud storage. Additionally, cross-region disaster recovery is implemented using Amazon S3 with S3 Cross-Region Replication and AWS Backup cross-region copy.

STEP 1: Launch EC2 Instances

Navigate:

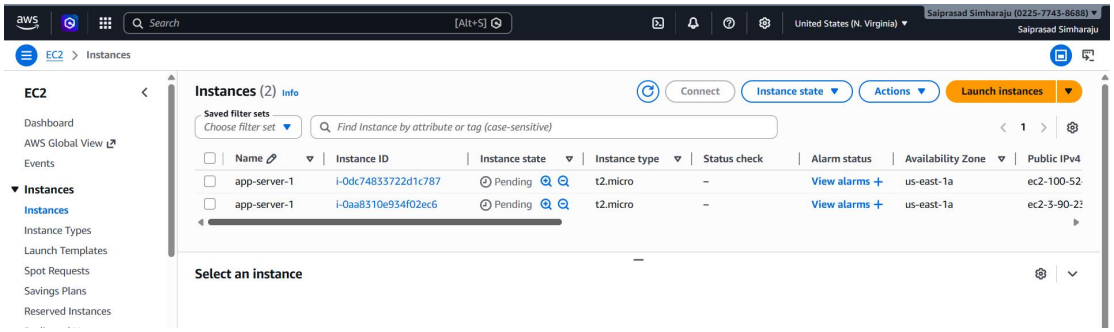
Go to → Amazon EC2
Click → Launch Instance

Configuration:

1. Name: AppServer1 and AppServer2
2. AMI: Amazon Linux 2
3. Instance type: t2.micro (or t3)
4. Network:
   Select your VPC
   Enable Autoassign Public IP
5. Security Group:
   Allow SSH (22)
   Allow HTTP (80)

Launch 2 instances (needed for MultiAttach demo)

Screenshots:



STEP 2: Create & Attach EBS Volume (MultiAttach)

Navigate:

EC2 → Volumes

Steps:

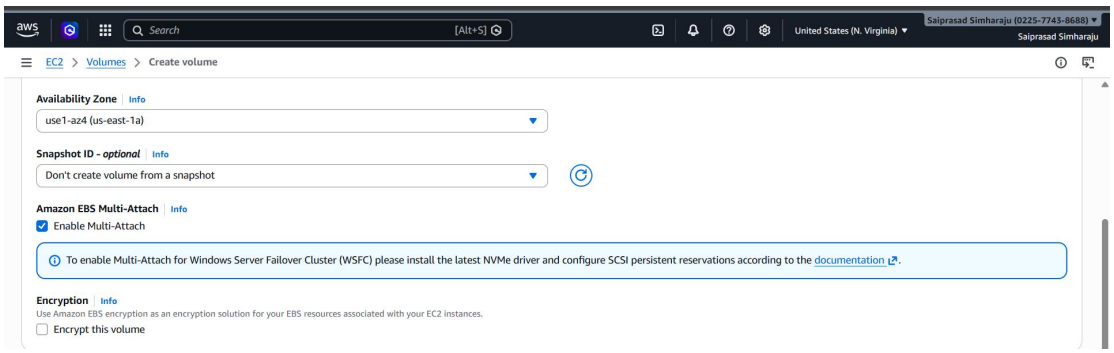
1. Click Create Volume
2. Configure:
   Type: io2
   Size: 100 GB
   Enable → MultiAttach
   Same AZ as EC2
3. Click Create

Attach Volume to Both Instances

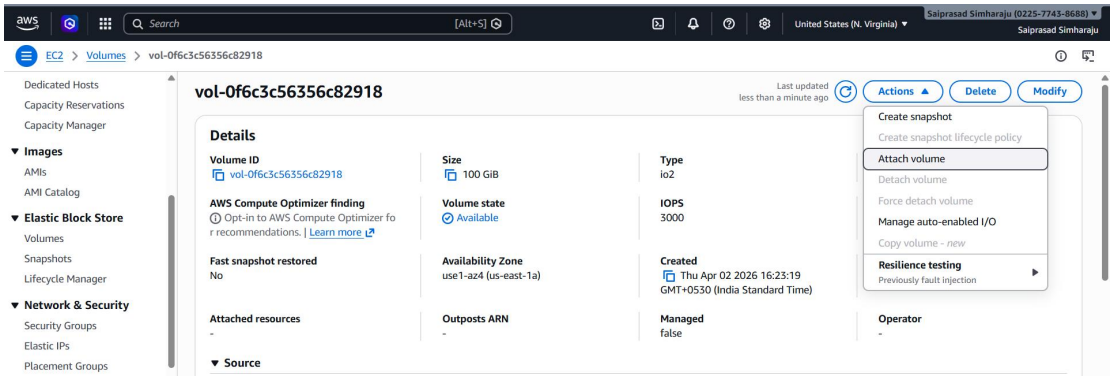
1. Select volume → Actions → Attach
2. Attach to:
   AppServer1
3. Repeat → Attach to:
   AppServer2

Uses Amazon EBS

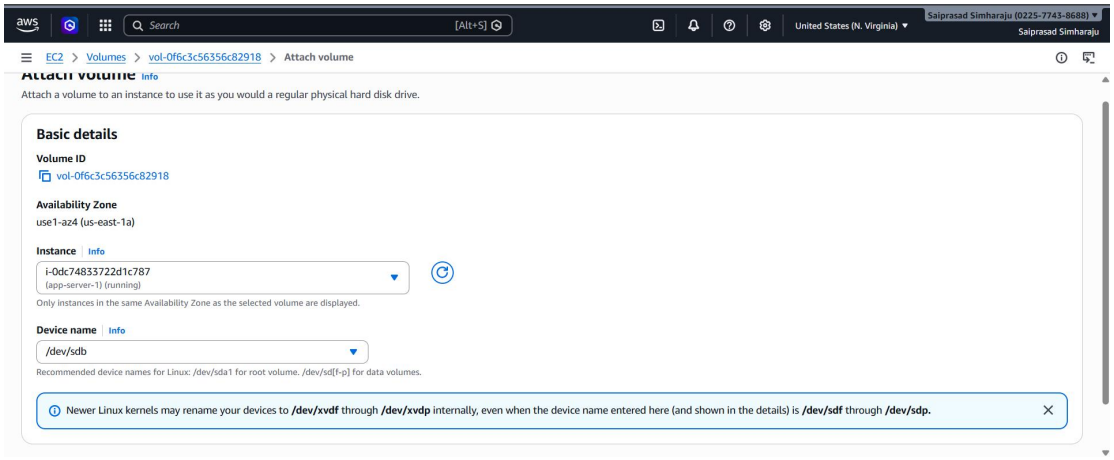
Screenshots:



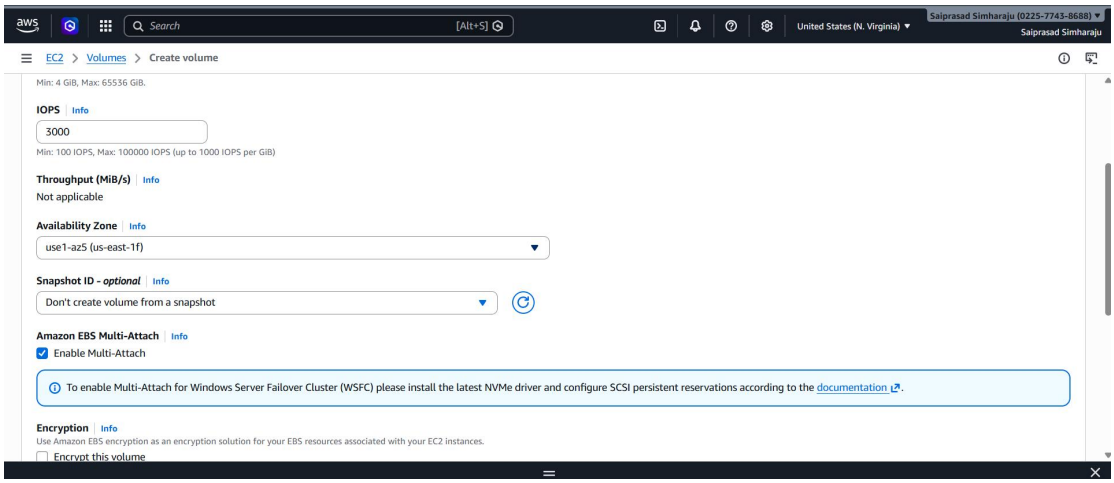
Attach volume



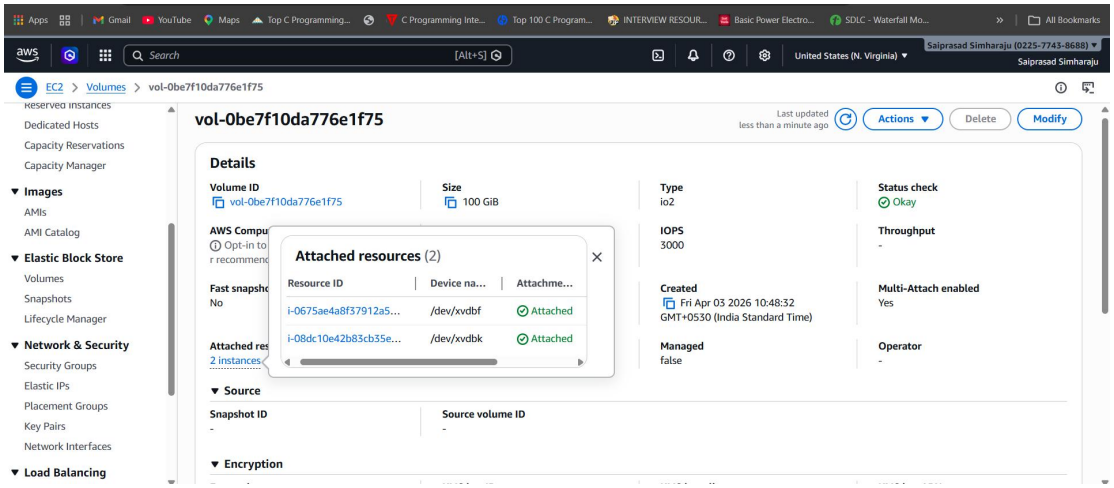
Attach volume to ec2



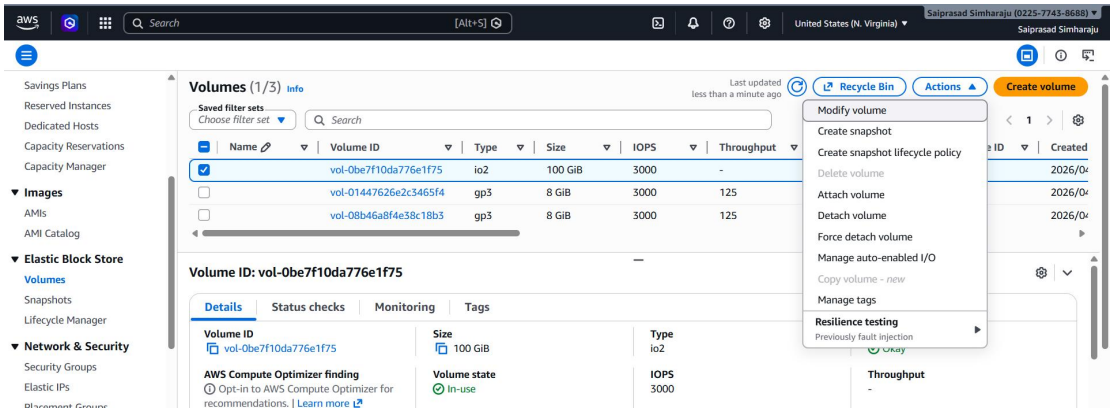
Enable Multi-Attach



List of Insatnes Attached to VOLUME



Click on Modify Option



STEP 3: Mount EBS Volume (on EC2)

SSH into instance:

```
bash id="jz7k2a"
ssh ec2user@<publicip>
```

Run:

```
bash id="l2x9qd"
lsblk
sudo mkfs t ext4 /dev/xvdf
sudo mkdir /data
sudo mount /dev/xvdbf /data
```

Repeat mount on second instance (without formatting)
Instance 1
Screenshots:

Do lsblk afterr connection to instance through ssh

```
ubuntu@ip-172-31-66-79:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0         7:0    0 27.8M  1 loop /snap/amazon-ssm-agent/12322
loop1         7:1    0 48.1M  1 loop /snap/snapd/25935
loop2         7:2    0   74M  1 loop /snap/core22/2339
nvme0n1     259:0    0    8G  0 disk
├─nvme0n1p1 259:1    0    7G  0 part /
├─nvme0n1p14
            259:2    0    4M  0 part
├─nvme0n1p15
            259:3    0  106M  0 part /boot/efi
└─nvme0n1p16
            259:4    0  913M  0 part /boot
nvme1n1     259:5    0  100G  0 disk
```

Make filesystem

```
ubuntu@ip-172-31-66-79:~$ sudo mkfs -t ext4 /dev/nvme1n1
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 26214400 4k blocks and 6553600 inodes
Filesystem UUID: bc0c55f1-c4cf-4718-b953-2c09d46b7fac
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
        4096000, 7962624, 11239424, 20480000, 23887872

Allocating group tables: done
Writing inode tables: done
Creating journal (131072 blocks): done
Writing superblocks and filesystem accounting information:   0/8done

ubuntu@ip-172-31-66-79:~$ sudo mkdir /data
sudo mount /dev/nvme1n1 /data
ubuntu@ip-172-31-66-79:~$ df -h
Filesystem       Size  Used Avail Use% Mounted on
/dev/root        6.8G  1.8G  4.9G  27% /
tmpfs            456M     0  456M   0% /dev/shm
tmpfs            183M  888K  182M   1% /run
tmpfs            5.0M     0  5.0M   0% /run/lock
efivarfs         128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16  881M   94M  726M  12% /boot
/dev/nvme0n1p15  105M  6.2M   99M   6% /boot/efi
tmpfs             92M   12K   92M   1% /run/user/1000
/dev/nvme1n1      98G   24K   93G   1% /data
```

Instance 2

```
ubuntu@ip-172-31-65-34:~$ lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0          7:0    0 27.8M  1 loop /snap/amazon-ssm-agent/12322
loop1          7:1    0   74M  1 loop /snap/core22/2339
loop2          7:2    0 48.1M  1 loop /snap/snapd/25935
nvme0n1      259:0    0    8G  0 disk
├─nvme0n1p1  259:1    0    7G  0 part /
├─nvme0n1p14 259:2    0    4M  0 part
├─nvme0n1p15 259:3    0  106M  0 part /boot/efi
└─nvme0n1p16 259:4    0  913M  0 part /boot
nvme1n1      259:5    0  100G  0 disk
```

Make filesystem for instance 2

```
ubuntu@ip-172-31-65-34:~$ sudo mkfs -t ext4 /dev/nvme1n1
mke2fs 1.47.0 (5-Feb-2023)
/dev/nvme1n1 contains a ext4 file system
        last mounted on Fri Apr  3 05:28:14 2026
Proceed anyway? (y,N) y
Creating filesystem with 26214400 4k blocks and 6553600 inodes
Filesystem UUID: d1f3ac14-dfd2-4fc9-874e-e6d669f2f06c
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
        4096000, 7962624, 11239424, 20480000, 23887872

Allocating group tables: done
Writing inode tables: done
Creating journal (131072 blocks): done
Writing superblocks and filesystem accounting information:   0/8done

ubuntu@ip-172-31-65-34:~$ sudo mkdir /data
sudo mount /dev/nvme1n1 /data
ubuntu@ip-172-31-65-34:~$ df -h
Filesystem       Size  Used Avail Use% Mounted on
/dev/root        6.8G  1.8G  4.9G  27% /
tmpfs            456M     0  456M   0% /dev/shm
tmpfs            183M  888K  182M   1% /run
tmpfs            5.0M     0  5.0M   0% /run/lock
efivarfs         128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16  881M   94M  726M  12% /boot
/dev/nvme0n1p15  105M  6.2M   99M   6% /boot/efi
tmpfs             92M   12K   92M   1% /run/user/1000
/dev/nvme1n1      98G   24K   93G   1% /data
ubuntu@ip-172-31-65-34:~$
```

Volume updated to 150 gb

```
ubuntu@ip-172-31-66-79:~$ lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0          7:0    0 27.8M  1 loop /snap/amazon-ssm-agent/12322
loop1          7:1    0 48.1M  1 loop /snap/snapd/25935
loop2          7:2    0   74M  1 loop /snap/core22/2339
nvme0n1      259:0    0    8G  0 disk
├─nvme0n1p1  259:1    0    7G  0 part /
├─nvme0n1p14 259:2    0    4M  0 part
├─nvme0n1p15 259:3    0  106M  0 part /boot/efi
└─nvme0n1p16 259:4    0  913M  0 part /boot
nvme1n1      259:5    0  150G  0 disk /data
ubuntu@ip-172-31-66-79:~$
```

Instance 2

```
ubuntu@ip-172-31-65-34:~$ lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0          7:0    0 27.8M  1 loop /snap/amazon-ssm-agent/12322
loop1          7:1    0   74M  1 loop /snap/core22/2339
loop2          7:2    0 48.1M  1 loop /snap/snapd/25935
nvme0n1      259:0    0    8G  0 disk
├─nvme0n1p1  259:1    0    7G  0 part /
├─nvme0n1p14 259:2    0    4M  0 part
├─nvme0n1p15 259:3    0  106M  0 part /boot/efi
└─nvme0n1p16 259:4    0  913M  0 part /boot
nvme1n1      259:5    0  150G  0 disk /data
ubuntu@ip-172-31-65-34:~$
```

Instance 2 150 bg

STEP 4: Resize EBS Volume (100GB → 150GB)

Console:

1. Select volume → Modify Volume
2. Change size → 150 GB

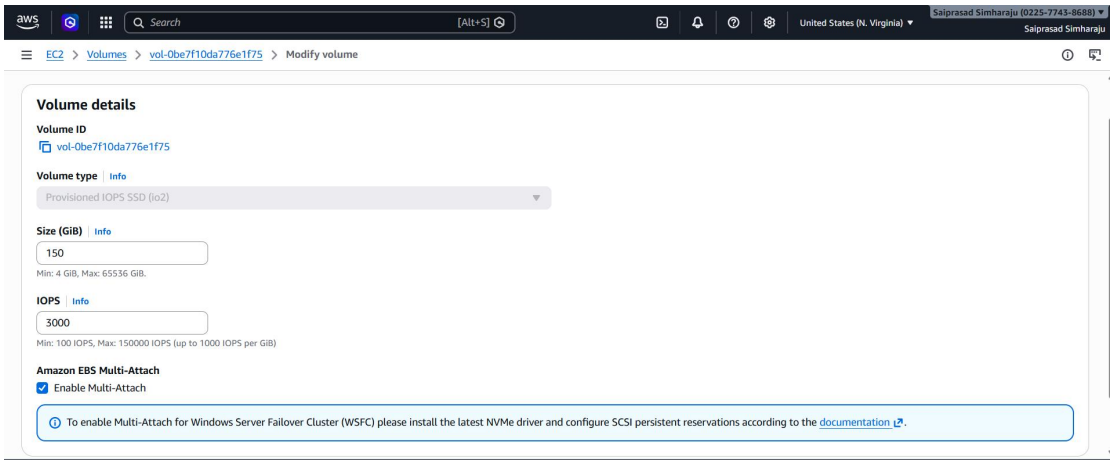
Instance Commands:

bash id="3f9dka"

lsblk
sudo growpart /dev/xvdf 1
sudo resize2fs /dev/xvdf1

Select 150 gb

Screenshots:




```
ubuntu@ip-172-31-65-34:~$ sudo resize2fs /dev/nvme1n1
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/nvme1n1 is mounted on /data; on-line resizing required
old_desc_blocks = 13, new_desc_blocks = 19
The filesystem on /dev/nvme1n1 is now 39321600 (4k) blocks long.

ubuntu@ip-172-31-65-34:~$ df -h
Filesystem       Size  Used Avail Use% Mounted on
/dev/root        6.8G  1.8G  4.9G  27% /
tmpfs            456M     0  456M   0% /dev/shm
tmpfs            183M  888K  182M   1% /run
tmpfs            5.0M     0  5.0M   0% /run/lock
efivarfs         128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16  881M   94M  726M  12% /boot
/dev/nvme0n1p15  105M  6.2M   99M   6% /boot/efi
tmpfs             92M   12K   92M   1% /run/user/1000
/dev/nvme1n1     148G   24K  141G   1% /data
ubuntu@ip-172-31-65-34:~$
```

STEP 5: Create EFS File System

Navigate:

Go to → Amazon EFS

Steps:

1. Click Create file system
2. Name: EnterpriseEFS
3. Select VPC
4. Create

Mount EFS on EC2

Install:

bash id="0tx6qv"

sudo yum install y amazonefsutils

Mount:

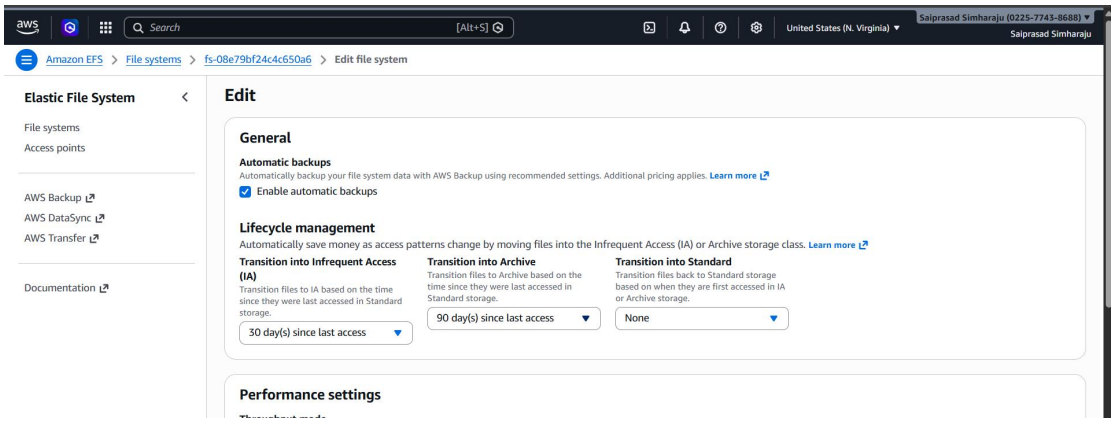
bash id="yq2p8m"
sudo mkdir /efs
sudo mount t efs fsxxxx:/ /efs

Shared storage across both instances

Lifecycle Policy:

1. Open EFS → Lifecycle
2. Set:
   IA → 30 days
   Archive → 90 days

Creating File System

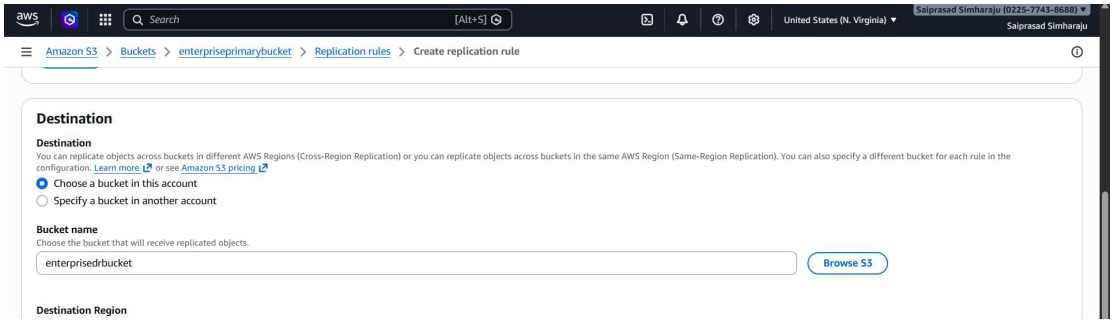


STEP 6: Create S3 Buckets

Navigate:

Go to → Amazon S3

Primary Bucket:
Name: enterpriseprimarybucket
Region: useast1
Enable Versioning
DR Bucket
Name: enterprisedrbucket
Region: uswest2
Enable Versioning
Give S3 bucket names



STEP 7: Enable CrossRegion Replication

1. Open primary bucket
2. Go to → Management → Replication
3. Create rule:
   Destination → DR bucket
   Replicate all objects

Uses S3 CrossRegion Replication

STEP 8: Configure AWS Backup

Navigate:

Go to → AWS Backup

Create Backup Vault
Name: EnterpriseBackupVault
Enable KMS encryption

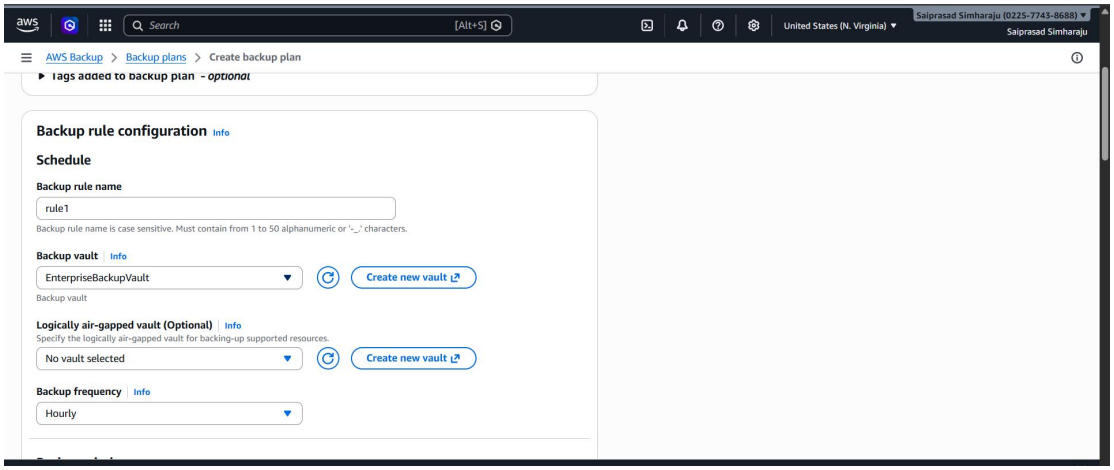
Create Backup Plan

1. Rule 1: Hourly EBS
   Frequency: Hourly
   Retention: 24 hrs
2. Rule 2: Daily EFS + AMI
   Frequency: Daily
   Retention: 30 days
3. Rule 3: CrossRegion Copy
   Destination: uswest2
   Retention: 1 year

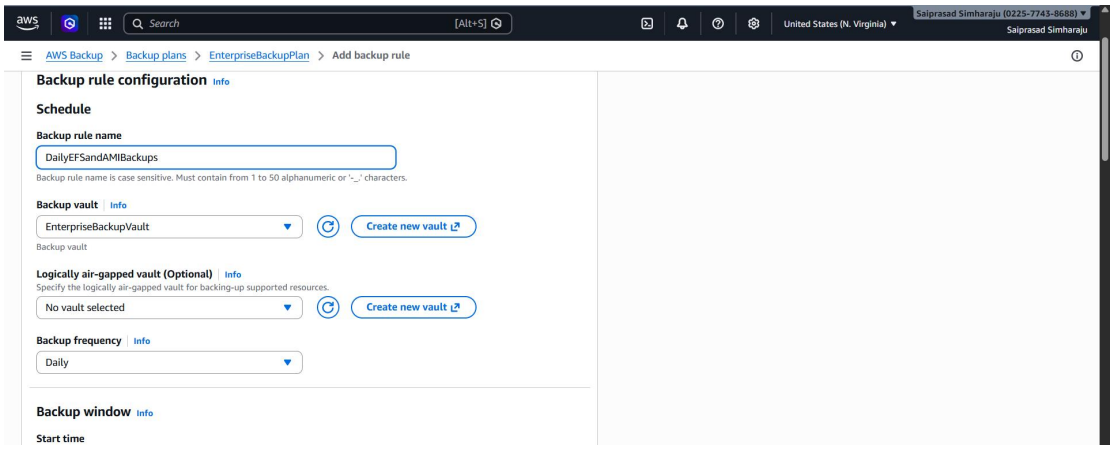
Assign Resources
Select:
EC2
EBS
EFS

Screenshots:

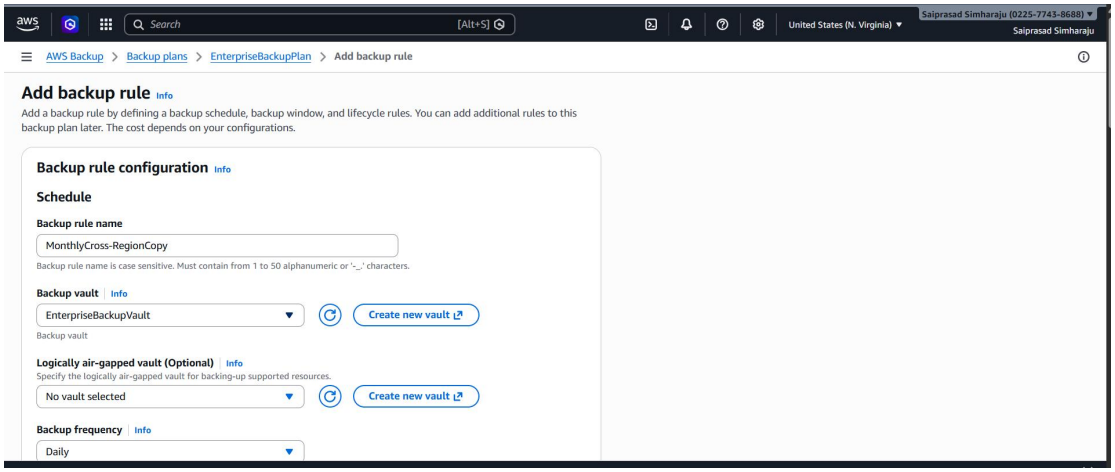
Rule 1

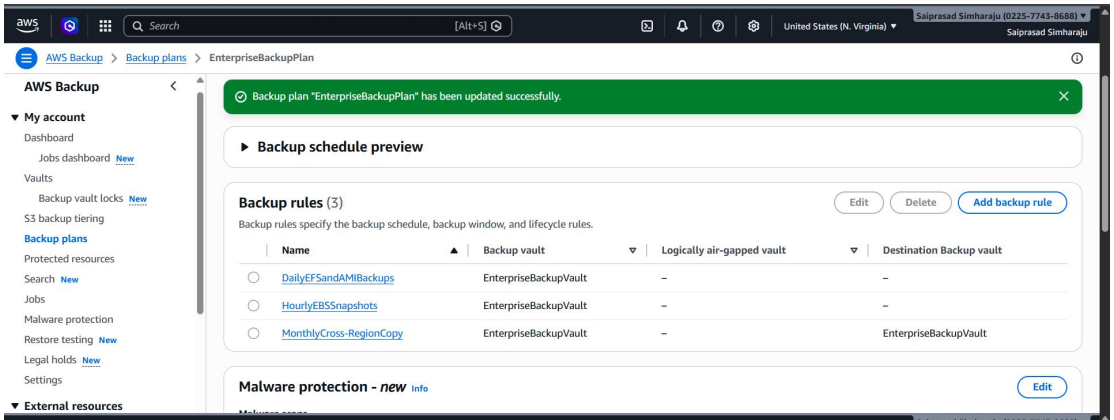


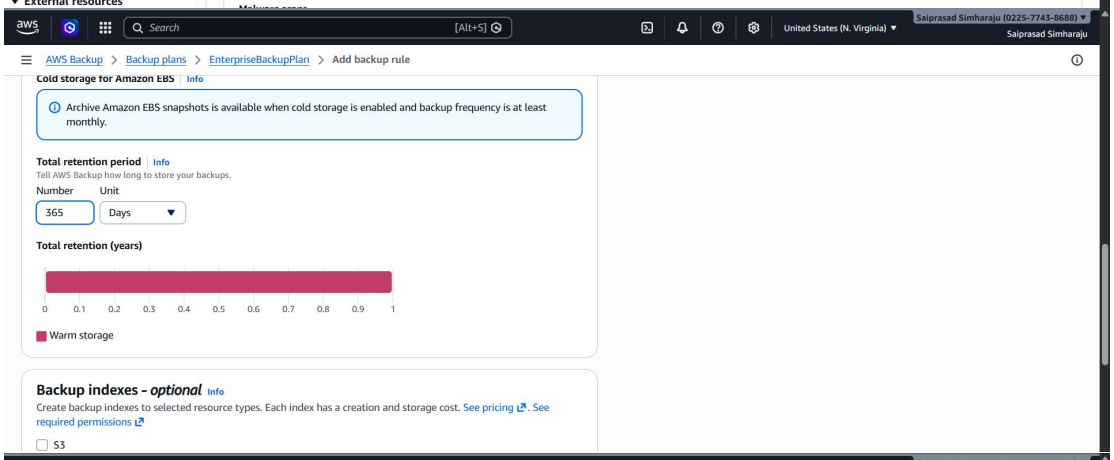
Rule 2

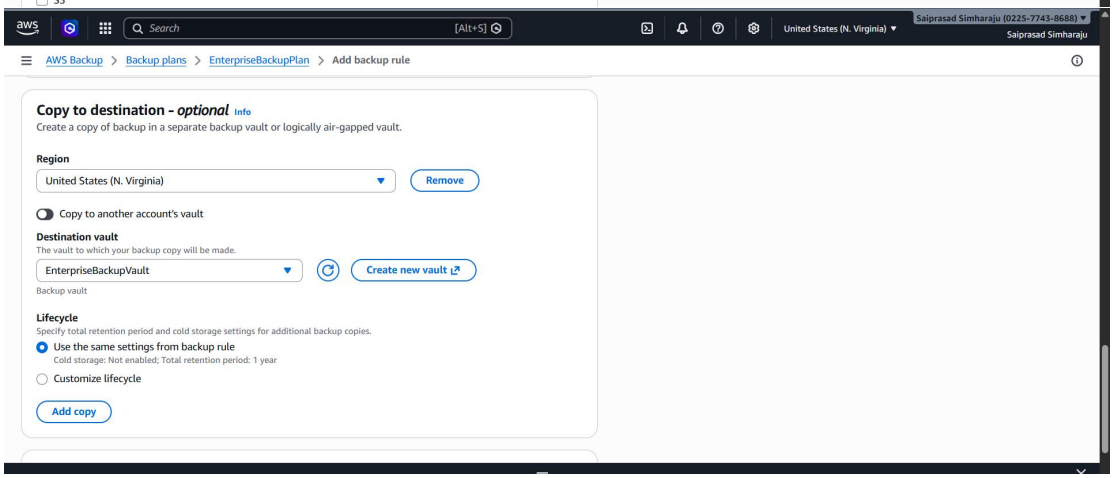


Rule 3

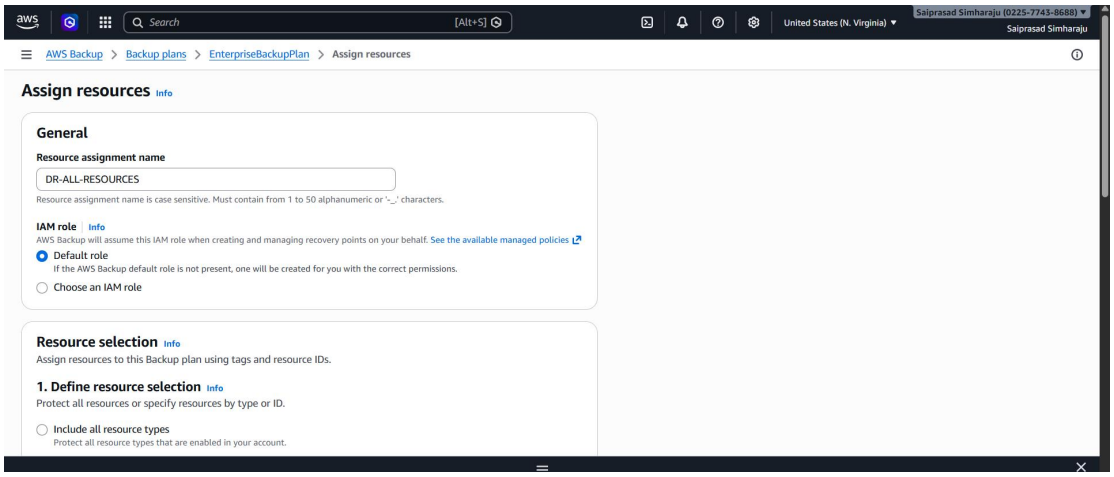


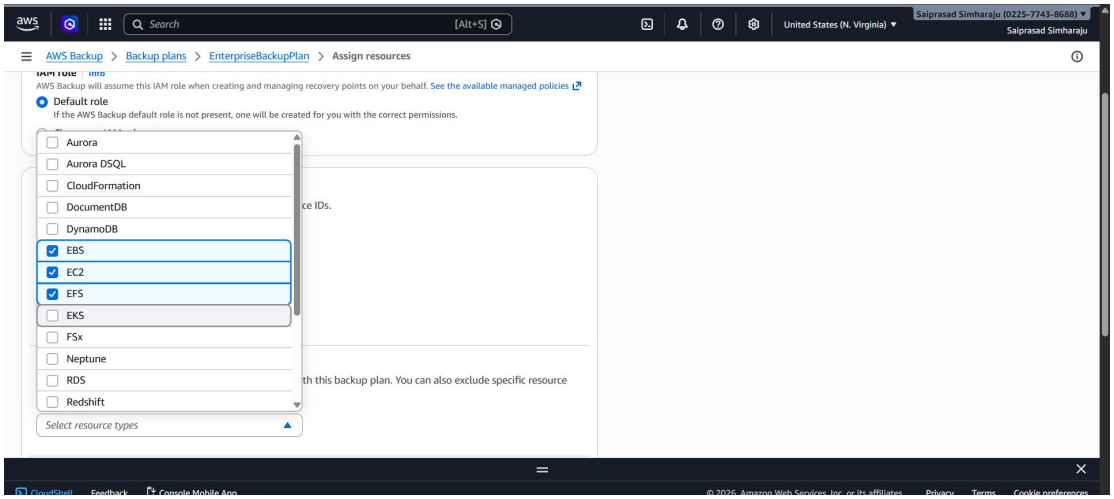




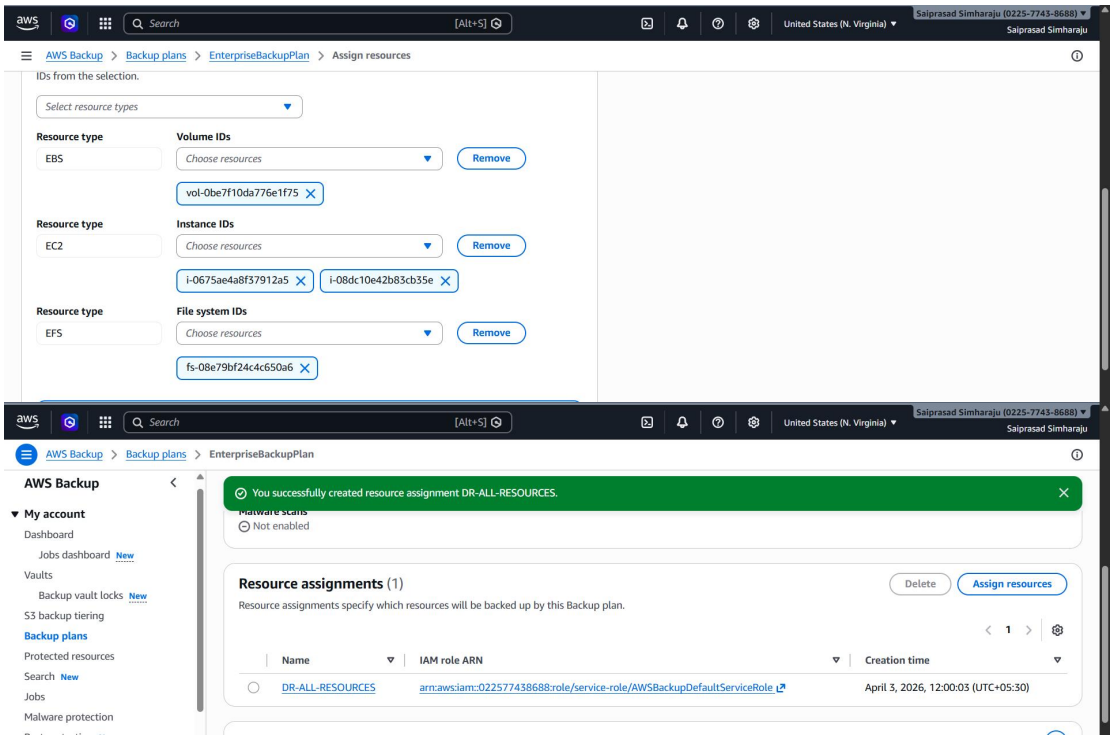


RESOURCES ALLOCATION





List of Resources Allocated



STEP 9: Setup Storage Gateway (Hybrid)

Navigate:

Go to → AWS Storage Gateway

Steps:

1. Click Create Gateway
2. Select:
   File Gateway
3. Deploy:
   As VM (onprem simulation)

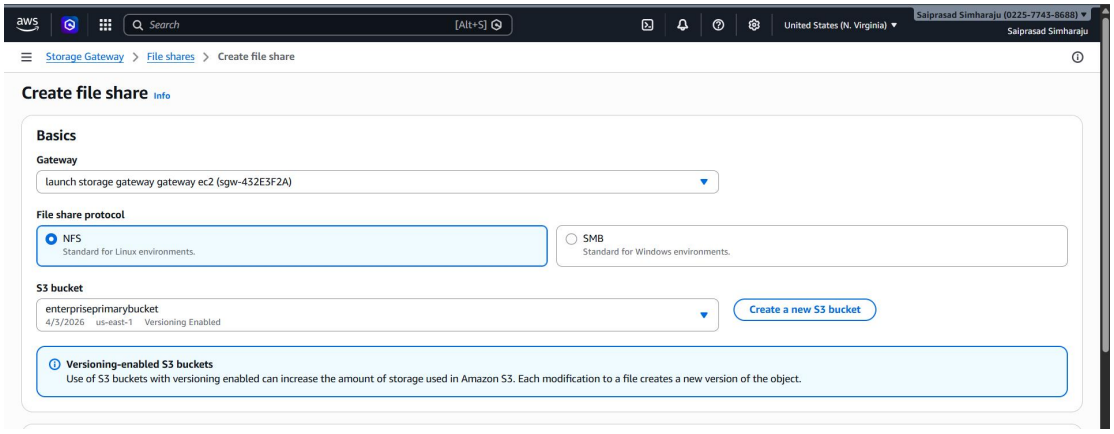
4. Configure:
   NFS / SMB share
5. Link to:

S3 primary bucket

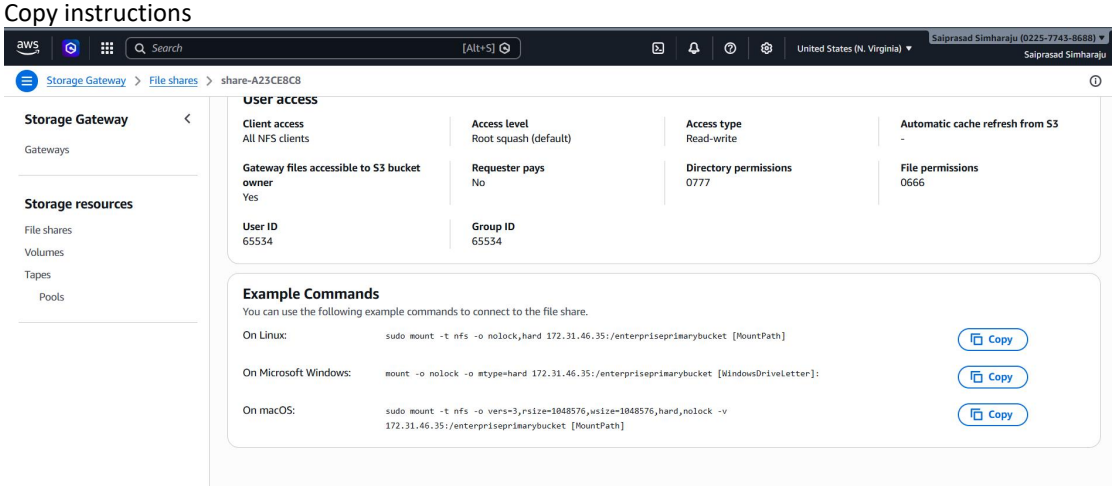
Onprem → AWS sync

Screenshots:

Nfs file share

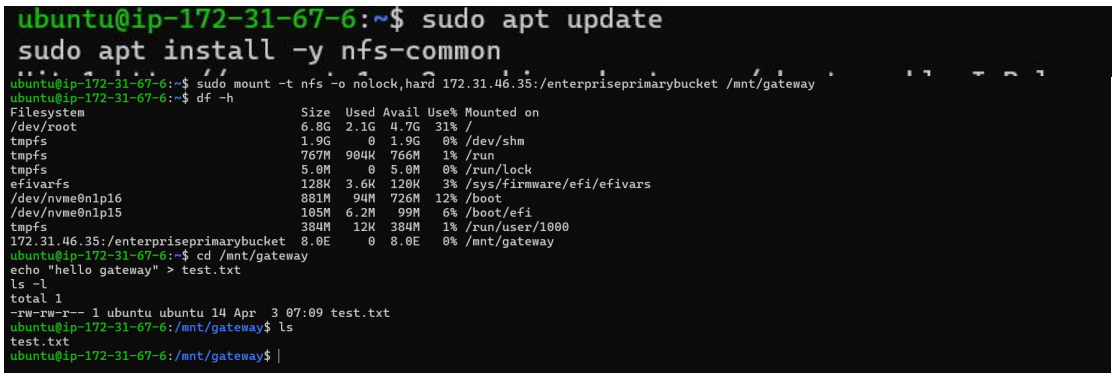


Copy instructions



Gateway

Create a file test.txt



STEP 10: Validate DR Architecture

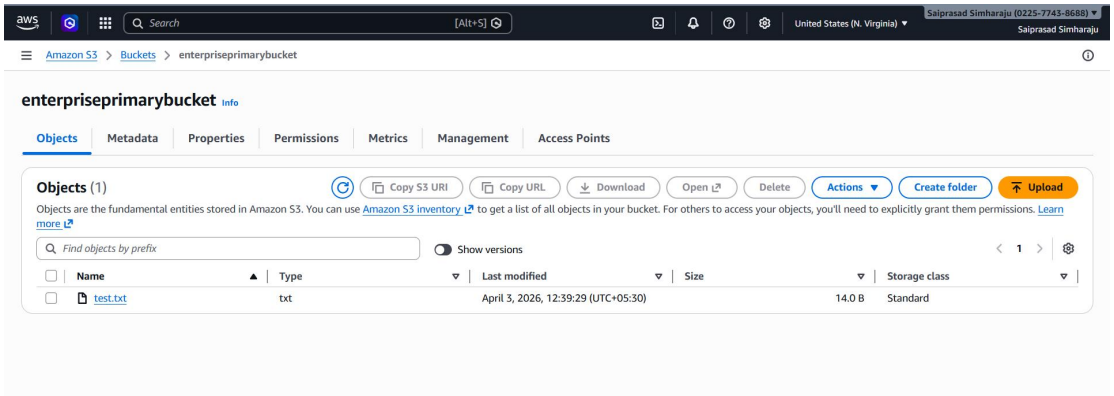
Test scenarios:

- Upload file → S3 → Check DR bucket replication
- Create file in EFS → Verify backup in AWS Backup
- Stop EC2 → Restore using AMI
- Check EBS snapshots

Screenshots:

```
ubuntu@ip-172-31-67-6:~$ sudo mount -t nfs -o nolock,hard 172.31.46.35:/enterpriseprimarybucket /mnt/gateway
ubuntu@ip-172-31-67-6:~$ df -h
Filesystem                              Size  Used Avail Use% Mounted on
/dev/root                               6.8G  2.1G  4.7G  31% /
tmpfs                                   1.9G     0  1.9G   0% /dev/shm
tmpfs                                   767M  904K  766M   1% /run
tmpfs                                   5.0M     0  5.0M   0% /run/lock
efivarfs                                128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16                         881M   94M  726M  12% /boot
/dev/nvme0n1p15                         105M  6.2M   99M   6% /boot/efi
tmpfs                                   384M   12K  384M   1% /run/user/1000
172.31.46.35:/enterpriseprimarybucket  8.0E     0  8.0E   0% /mnt/gateway
ubuntu@ip-172-31-67-6:~$ cd /mnt/gateway
echo "hello gateway" > test.txt
ls -l
total 1
-rw-rw-r-- 1 ubuntu ubuntu 14 Apr  3 07:09 test.txt
ubuntu@ip-172-31-67-6:/mnt/gateway$ ls
test.txt
ubuntu@ip-172-31-67-6:/mnt/gateway$
```

Validation
Storage Gateway → Source Bucket → (CRR) → DR Bucket



FINAL ARCHITECTURE FLOW:

EC2 (App Servers)
↓
EBS (MultiAttach)
↓
EFS (Shared Storage)

↓
AWS Backup → Vault → Crossregion copy

OnPrem Data
↓
Storage Gateway
↓
S3 (Primary)
↓ (CRR)
S3 (DR Region)

HOW THIS MEETS REQUIREMENTS

Requirement	Implementation
RPO = 1 hr	Hourly EBS backup
RTO = 4 hr	AMI + crossregion recovery
Shared storage	EFS
Hybrid backup	Storage Gateway
DR	S3 CRR + Backup copy